

Relazione per il corso Basi di Dati

A.A 2023/2024

Progetto di una base di dati per la gestione di una
palestra

Marco Antolini
marco.antolini6@studio.unibo.it
0000977047

Analisi dei requisiti

1. Intervista	4
2. Rilevamento ambiguità e correzioni proposte.....	5
3. Principali azioni richieste	6

Progettazione concettuale

1. Schema scheletro	8
2. Raffinamenti proposti.....	10
3. Schema concettuale finale	11

Progettazione logica

1. Stima del volume dei dati.....	13
2. Descrizione delle operazioni principali e stima della loro frequenza.....	14
3. Schemi di navigazione e tabelle degli accessi.....	15
4. Raffinamento dello schema.....	16
5. Analisi delle ridondanze	17
6. Traduzione di entità e associazioni in relazioni.....	18
7. Schema relazionale finale	19
8. Traduzione in linguaggio SQL	20

Progettazione dell'applicazione

1. Descrizione dell'architettura dell'applicazione realizzata.....	26
--	----

Analisi dei requisiti

1. Intervista

L'obiettivo del progetto è creare un software gestionale per una palestra, principalmente per quanto riguarda gli aspetti economici.

Saranno presenti due tipi di utenti, amministratore e dipendente, il primo con accesso a tutti i dati, mentre il secondo con accesso limitato.

Nel database si terrà traccia di tutti gli utenti (quindi inclusi sia i dipendenti che gli admin): verrà memorizzato il loro codice identificativo, i dati anagrafici, un recapito telefonico o di posta elettronica e la data di ammissione.

Ogni membro della staff dovrà avere un account con il quale poter accedere all'applicativo.

Per ogni dipendente il database terrà anche traccia del suo contratto di lavoro (ed eventuali contratti passati) e delle sue timbrature (all'entrata e uscita del posto di lavoro). Sarà quindi possibile per un admin gestire il contratto degli altri dipendenti, mentre ogni utente sarà in grado di registrare le timbrature di un dipendente (cosa che idealmente accadrebbe in realtà tramite lettura del cartellino).

Il database dovrà tenere traccia di tutti i pagamenti in uscita, che siano questi il pagamento di uno stipendio, l'acquisto di attrezzatura utile, saldo di una bolletta o pagamento di un intervento (come potrebbe essere ad esempio una manutenzione).

Per quanto riguarda i clienti il database dovrà nuovamente memorizzare il loro codice identificativo, i dati anagrafici, un recapito telefonico o di posta elettronica e la data di iscrizione alla palestra.

I clienti potranno acquistare abbonamenti o pacchetti di entrate, i quali a loro volta verranno memorizzati nel database oltre ai listini con le opzioni e i prezzi di ogni anno.

Un cliente con un abbonamento attivo o delle entrate ancora disponibili potrà accedere alla palestra (in caso avesse delle entrate disponibili ma anche un abbonamento attivo, non verranno scalate le entrate ma verrà considerato l'abbonamento come metodo di ingresso).

Infine verrà tenuta traccia anche di tutte le vendite di abbonamenti e pacchetti entrate nei confronti dei clienti.

Sono incluse anche alcune query statistiche quali:

- calcolare il guadagno degli impiegati (e quindi la spesa sostenuta) in un certo periodo
- visualizzare la frequenza di entrata di clienti nella palestra (giornaliera, settimanale o mensile) in un determinato periodo

2. Rilevamento ambiguità e correzioni proposte

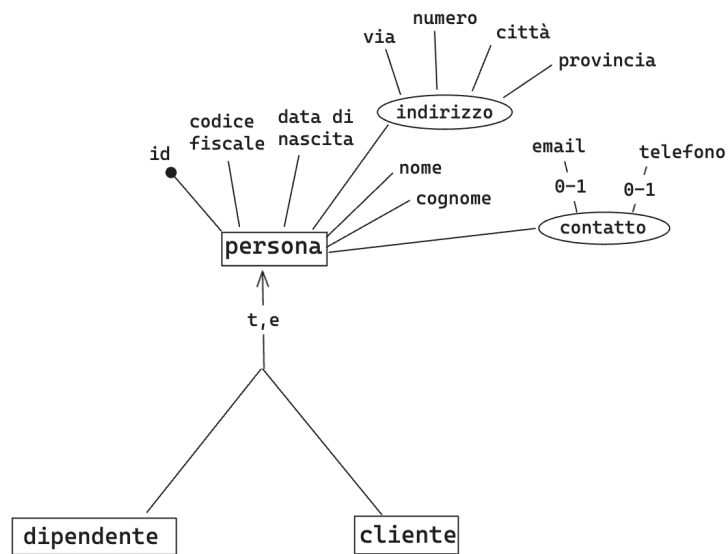
3. Principali azioni richieste

Si richiedono le seguenti funzionalità principali:

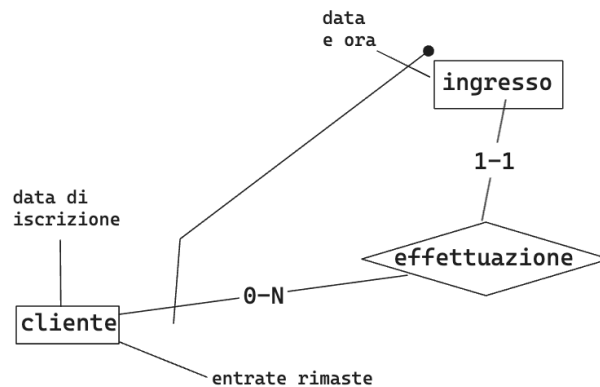
- Inserimento, modifica e visualizzazione di dipendenti
- Inserimento, modifica e visualizzazione degli account dei dipendenti
- Inserimento, modifica e visualizzazione dei contratti di lavoro
- Registrazione e visualizzazione di entrata e uscita dei dipendenti dal posto di lavoro
- Inserimento e visualizzazione dei pagamenti effettuati
- Inserimento, modifica e visualizzazione di clienti
- Registrazione e visualizzazione di prodotti acquistati (abbonamenti e pacchetti entrate)
- Inserimento, modifica e visualizzazione dei listini prezzi
- Registrazione e visualizzazione dell'entrata dei clienti in palestra
- Visualizzazione degli acquisti effettuati dai clienti
- Visualizzazione entrate e uscite economiche in un determinato periodo
- Visualizzazione del numero di entrate dei clienti in base al momento della giornata, giorno della settimana o periodo dell'anno
- Visualizzazione del guadagno dei dipendenti in un determinato periodo

Progettazione concettuale

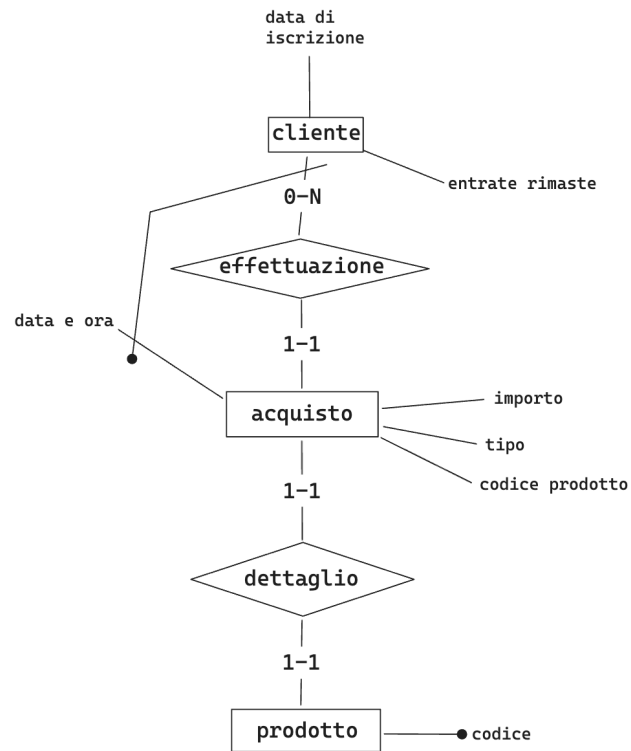
1. Schema scheletro



L'entità di base Persona viene generalizzata in due entità Dipendente e Cliente che sono le due entità principali del dominio.



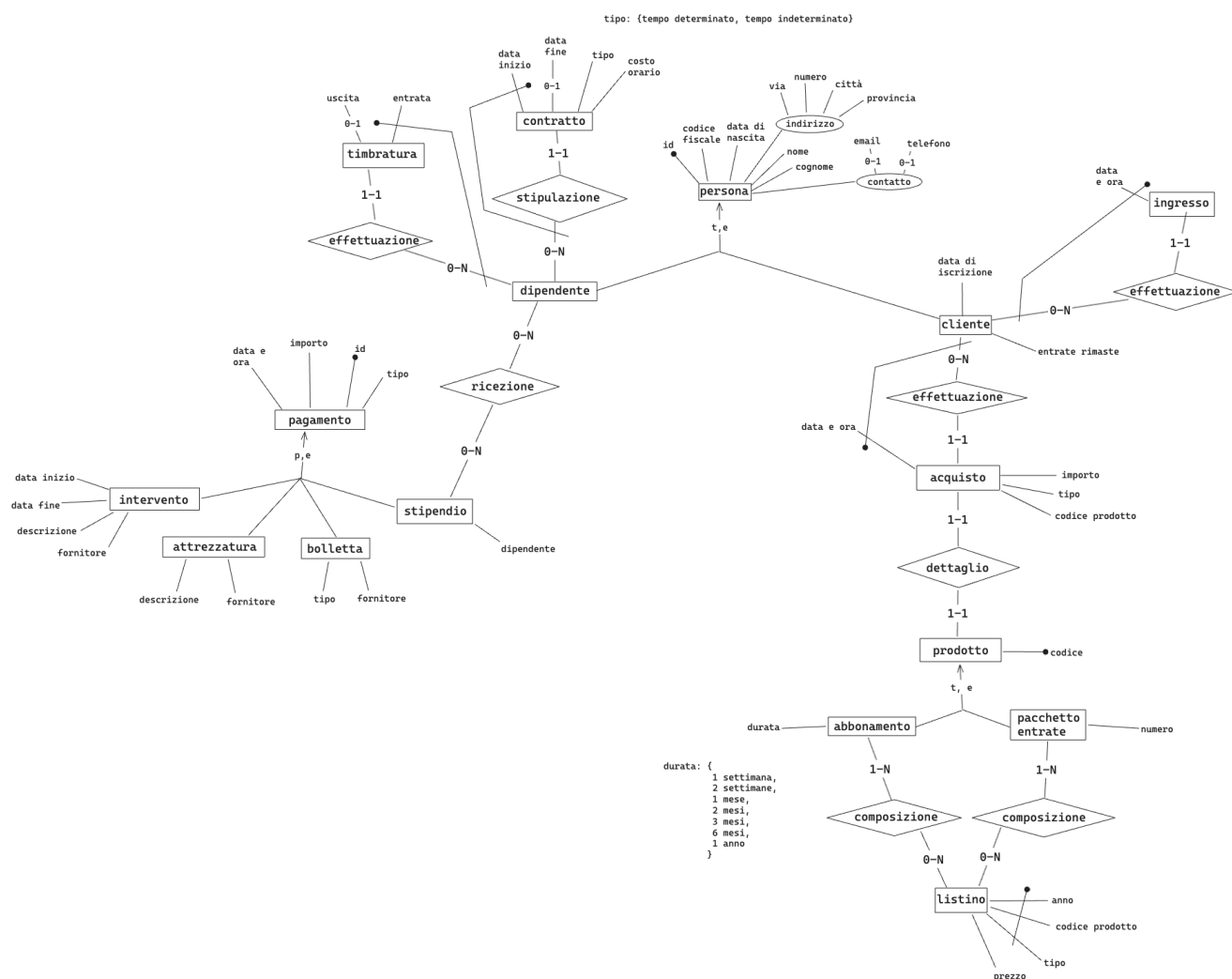
Ogni cliente può entrare nella palestra quando desidera e gli ingressi vengono registrati tramite i rispettivi data e orario e il codice del cliente coinvolto. Si considera possibile per un cliente rientrare nella stessa giornata e per questo viene salvato anche l'orario di entrata.



Ogni cliente per entrare nella palestra necessita di acquistare un prodotto

2. Raffinamenti proposti

3. Schema concettuale finale



Progettazione logica

1. Stima del volume dei dati

Concetto	Tipo	Volume
Account	E	
Dipendente	E	
Contratto	E	
Timbratura	E	
Stipendio	E	
Attrezzatura	E	
Bolletta	E	
Intervento	E	
Pagamento	E	
Cliente	E	
Ingresso	E	
Acquisto	E	
Prodotto	E	
Abbonamento	E	
Pacchetto Ingressi	E	
Listino	E	
Effettuazione	R	
	R	
	R	
	R	
	R	
	R	
	R	
	R	

2. Descrizione delle operazioni principali e stima della loro frequenza

3. Schemi di navigazione e tabelle degli accessi

4. Raffinamento dello schema

4.1. Eliminazione delle gerarchie

4.2. Eliminazione degli attributi composti

4.3. Scelta delle chiavi primarie

4.4. Eliminazione degli identificatori esterni

5. Analisi delle ridondanze

6. Traduzione di entità e associazioni in relazioni

ACCOUNT(id_dipendente: dipendenti, username, password, ruolo, approvato)

DIPENDENTI(id, codice_fiscale, nome, cognome, data_nascita, via, civico, città, provincia, telefono, email, data_assunzione)

CONTRATTI(id_dipendente: dipendenti, tipo, costo_orario, data_inizio, data_fine)

TIMBRATURE(id_dipendente: dipendenti, entrata, uscita)

STIPENDI(id_pagamento: pagamenti, id_dipendente: dipendenti)

ATTREZZATURE(id_pagamento: pagamenti, descrizione, fornitore)

BOLLETTE(id_pagamento: pagamenti, descrizione, fornitore)

INTERVENTI(id_pagamento: pagamenti, descrizione, attuatore, data_inizio, data_fine)

PAGAMENTI(id, data, importo, tipo)

CLIENTI(id, codice_fiscale, nome, cognome, data_nascita, via, civico, città, provincia, telefono, email, data_iscrizione, ingressi_rimanenti)

INGRESSI(id_cliente: clienti, data)

ACQUISTI(id_cliente: clienti, data, importo, tipo, codice_prodotto)

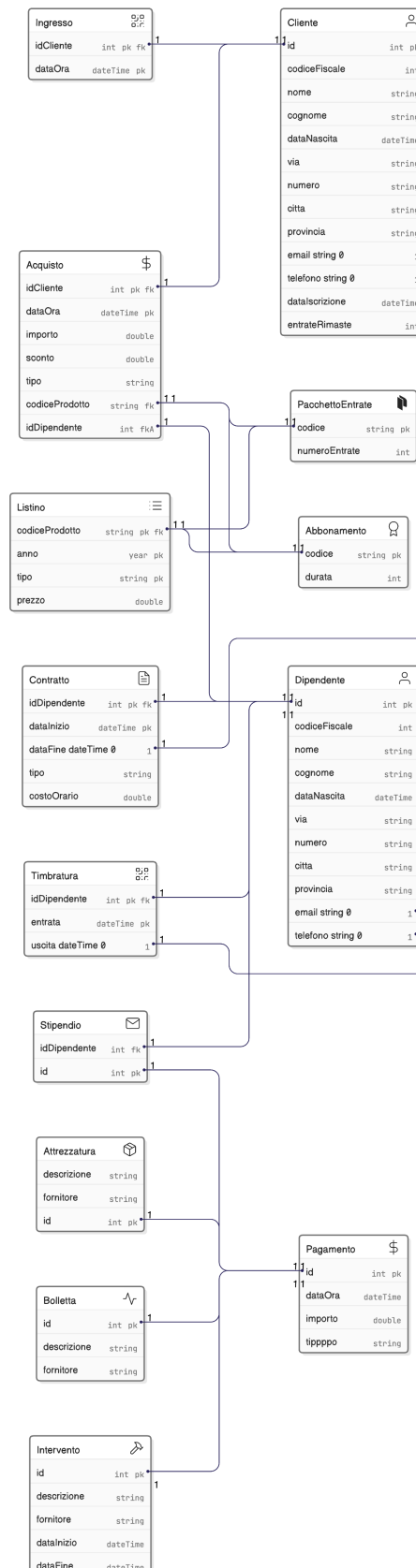
PRODOTTI(codice)

ABBONAMENTI(codice_prodotto: prodotti, durata)

PACCHETTI_INGRESSI(codice_prodotto: prodotti, numero_ingressi)

LISTINI(anno, tipo, codice_prodotto, prezzo)

7. Schema relazionale finale



8. Traduzione in linguaggio SQL

8.1. Costruzione delle tabelle

```
CREATE TABLE account (  
    username VARCHAR(255) PRIMARY KEY,  
    password VARCHAR(255) NOT NULL,  
    ruolo ENUM('Admin', 'Dipendente') DEFAULT 'Dipendente',  
    approvato BOOLEAN DEFAULT FALSE,  
    id_dipendente INT UNIQUE,  
    CONSTRAINT fk_account_dipendenti FOREIGN KEY (id_dipendente)  
REFERENCES dipendenti(id) ON DELETE CASCADE  
);
```

```
CREATE TABLE dipendenti (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    codice_fiscale VARCHAR(255) UNIQUE NOT NULL,  
    nome VARCHAR(255) NOT NULL,  
    cognome VARCHAR(255) NOT NULL,  
    data_nascita DATE NOT NULL,  
    via VARCHAR(255) NOT NULL,  
    civico VARCHAR(10) NOT NULL,  
    città VARCHAR(255) NOT NULL,  
    provincia VARCHAR(255) NOT NULL,  
    telefono VARCHAR(20) NOT NULL,  
    email VARCHAR(255) NOT NULL,  
    data_assunzione DATE DEFAULT CURRENT_DATE  
);
```

```
CREATE TABLE contratti (  
    id_dipendente INT NOT NULL,  
    tipo ENUM('Tempo determinato', 'Tempo indeterminato') NOT NULL,  
    costo_orario FLOAT NOT NULL,  
    data_inizio DATE NOT NULL,  
    data_fine DATE,  
    PRIMARY KEY (id_dipendente, data_inizio),  
    CONSTRAINT fk_contratti_dipendenti FOREIGN KEY (id_dipendente)  
REFERENCES dipendenti(id) ON DELETE CASCADE  
);
```

```

CREATE TABLE timbrature (
    id_dipendente INT NOT NULL,
    entrata DATETIME NOT NULL,
    uscita DATETIME,
    PRIMARY KEY (id_dipendente, entrata),
    CONSTRAINT fk_timbrature_dipendenti FOREIGN KEY (id_dipendente)
REFERENCES dipendenti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE pagamenti (
    id INT AUTO_INCREMENT PRIMARY KEY,
    data DATE DEFAULT CURRENT_DATE,
    importo FLOAT NOT NULL,
    tipo ENUM('Stipendio', 'Bolletta', 'Attrezzatura', 'Intervento') NOT NULL
);

```

```

CREATE TABLE stipendi (
    id_pagamento INT PRIMARY KEY,
    id_dipendente INT NOT NULL,
    CONSTRAINT fk_stipendi_pagamenti FOREIGN KEY (id_pagamento)
REFERENCES pagamenti(id) ON DELETE CASCADE,
    CONSTRAINT fk_stipendi_dipendenti FOREIGN KEY (id_dipendente)
REFERENCES dipendenti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE attrezzature (
    id_pagamento INT PRIMARY KEY,
    descrizione TEXT NOT NULL,
    fornitore VARCHAR(255) NOT NULL,
    CONSTRAINT fk_attrezzature_pagamenti FOREIGN KEY (id_pagamento)
REFERENCES pagamenti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE bollette (
    id_pagamento INT PRIMARY KEY,
    descrizione TEXT NOT NULL,
    fornitore VARCHAR(255) NOT NULL,
    CONSTRAINT fk_bollette_pagamenti FOREIGN KEY (id_pagamento)
REFERENCES pagamenti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE interventi (
    id_pagamento INT PRIMARY KEY,
    descrizione TEXT NOT NULL,
    attuatore VARCHAR(255) NOT NULL,
    data_inizio DATETIME NOT NULL,
    data_fine DATETIME NOT NULL,
    CONSTRAINT fk_interventi_pagamenti FOREIGN KEY (id_pagamento)
REFERENCES pagamenti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE clienti (
    id INT AUTO_INCREMENT PRIMARY KEY,
    codice_fiscale VARCHAR(255) UNIQUE NOT NULL,
    nome VARCHAR(255) NOT NULL,
    cognome VARCHAR(255) NOT NULL,
    data_nascita DATE NOT NULL,
    via VARCHAR(255) NOT NULL,
    civico VARCHAR(10) NOT NULL,
    città VARCHAR(255) NOT NULL,
    provincia VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    email VARCHAR(255) NOT NULL,
    data_iscrizione DATE DEFAULT CURRENT_DATE,
    ingressi_rimanenti INT NOT NULL
);

```

```

CREATE TABLE ingressi (
    id_cliente INT NOT NULL,
    data DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_cliente, data),
    CONSTRAINT fk_ingressi_clienti FOREIGN KEY (id_cliente) REFERENCES
clienti(id) ON DELETE CASCADE
);

```

```

CREATE TABLE acquisti (
    id_cliente INT NOT NULL,
    data DATETIME DEFAULT CURRENT_TIMESTAMP,
    importo FLOAT NOT NULL,
    tipo ENUM('Abbonamento', 'Pacchetto ingressi') NOT NULL,
    codice_prodotto VARCHAR(255) NOT NULL,
    PRIMARY KEY (id_cliente, data),
    CONSTRAINT fk_acquisti_clienti FOREIGN KEY (id_cliente) REFERENCES
clienti(id) ON DELETE CASCADE,
    CONSTRAINT fk_acquisti_prodotti FOREIGN KEY (codice_prodotto)
REFERENCES prodotti(codice) ON DELETE CASCADE
);

```

```

CREATE TABLE prodotti (
    codice VARCHAR(255) PRIMARY KEY
);

```

```

CREATE TABLE abbonamenti (
    codice_prodotto VARCHAR(255) PRIMARY KEY,
    durata INT NOT NULL,
    CONSTRAINT fk_abbonamenti_prodotti FOREIGN KEY (codice_prodotto)
REFERENCES prodotti(codice) ON DELETE CASCADE
);

```

```

CREATE TABLE pacchetti_ingressi (
    codice_prodotto VARCHAR(255) PRIMARY KEY,
    numero_ingressi INT NOT NULL,
    CONSTRAINT fk_pacchetti_prodotti FOREIGN KEY (codice_prodotto)
REFERENCES prodotti(codice) ON DELETE CASCADE
);

```

```

CREATE TABLE listini (
    anno INT NOT NULL,
    tipo ENUM('Abbonamento', 'Pacchetto ingressi') NOT NULL,
    codice_prodotto VARCHAR(255) NOT NULL,
    prezzo FLOAT NOT NULL,
    PRIMARY KEY (anno, tipo, codice_prodotto),
    CONSTRAINT fk_listini_prodotti FOREIGN KEY (codice_prodotto)
REFERENCES prodotti(codice) ON DELETE CASCADE
);

```

8.2. Traduzione delle operazioni

Progettazione dell'applicazione

1. Descrizione dell'architettura dell'applicazione realizzata

L'applicazione è stata realizzata in linguaggio Typescript utilizzando il framework NextJS (che si basa su React, una libreria per interfacce web) e Prisma ORM come strumento per interfacciarsi al database mySQL (che risiede in locale).

Dopo una prima fase di registrazione e login l'utente si trova di fronte a una dashboard che può essere navigata per visualizzare le varie tabelle e compiere le rispettive operazioni.

La grafica del gestionale sarà differente a seconda del ruolo dell'utente: un admin avrà accesso a tutte le tabelle e operazioni mentre un semplice dipendente avrà un accesso limitato.

Operazioni come ad esempio registrare l'ingresso di un cliente in palestra o la timbratura di un dipendente all'entrata e uscita del posto di lavoro sarebbero idealmente effettuate automaticamente tramite un apposito tesserino (con codice a barre o magnetico) ma in questo caso è stata anche aggiunta la possibilità di registrarle manualmente per testare l'effettivo corretto comportamento del software.

Per pura utilità sono stati aggiunti in ogni tabella alcuni possibili filtri da applicare e barre di ricerca, che però agiscono solo lato client per filtrare più precisamente i dati ottenuti dal database (queste azioni che non interrogano o modificano il db sono riconoscibili perché in ogni pagina si trovano nella seconda "corsia" di possibili azioni effettuabili).